Latest updates: https://dl.acm.org/doi/10.1145/3712255.3726682

POSTER

Solution Approaches for the Balanced Task Planning Problem

FABIAN NAGLER, Vienna University of Technology, Vienna, Vienna, Austria

NYSRET MUSLIU, Vienna University of Technology, Vienna, Vienna, Austria

FELIX WINTER, Vienna University of Technology, Vienna, Vienna, Austria

Open Access Support provided by:

Vienna University of Technology

PDF Download
3712255.3726682.pdf
13 March 2026
Total Citations: 0
Total Downloads: 441

Published: 11 August 2025

Citation in BibTeX format

GECCO '25 Companion: Genetic and Evolutionary Computation Conference Companion
July 14 - 18, 2025
Malaga, Spain

Conference Sponsors:
SIGEVO

Solution Approaches for the Balanced Task Planning Problem

Fabian Nagler
Christian Doppler Laboratory for Artificial Intelligence and Optimization for Planning and Scheduling, TU Wien, MCP GmbH
Vienna, Austria
fna@packaging-cockpit.com

Nysret Musliu
Christian Doppler Laboratory for Artificial Intelligence and Optimization for Planning and Scheduling, TU Wien
Vienna, Austria
nysret.musliu@tuwien.ac.at

Felix Winter
Christian Doppler Laboratory for Artificial Intelligence and Optimization for Planning and Scheduling, TU Wien
Vienna, Austria
felix.winter@tuwien.ac.at

Abstract

In this paper, we introduce and solve the Balanced Task Planning Problem, which originates from real-life industrial domains. The aim of this problem is to assign tasks to machines and planning periods so that machine loads are balanced around a target value while prioritizing more critical tasks. We provide a formal problem description and use Constraint Programming and Integer Programming solvers to find optimal solutions. For large-scale instances, we propose local search variants based on Tabu Search. To evaluate the methods, we generate a large number of random problem instances and perform an experimental evaluation of the different approaches. The results show that most of the small instances can be solved using the exact approach, and, for some, optimal solutions can be obtained. Using Tabu Search, high-quality solutions could be found for the large majority of large-scale instances.

CCS Concepts

• **Computing methodologies** → Search methodologies; • **Mathematics of computing** → Combinatorial optimization; • **Theory of computation** → Algorithm design techniques.

Keywords

Tabu Search, Task Planning, Exact Methods

ACM Reference Format:
Fabian Nagler, Nysret Musliu, and Felix Winter. 2025. Solution Approaches for the Balanced Task Planning Problem. In *Genetic and Evolutionary Computation Conference (GECCO '25 Companion), July 14–18, 2025, Malaga, Spain.* ACM, New York, NY, USA, 4 pages. https://doi.org/10.1145/3712255.3726682

1 Introduction

In the area of production planning, assigning tasks to machines or employees is a highly relevant problem. Solving such types of problems can be challenging, especially if they include a large number of constraints and objectives. However, finding high-quality solutions is of great importance for applications in many industrial domains. In this paper, we approach a combinatorial optimization problem that originates from the field of production planning: the Balanced Task Planning Problem (BTPP).

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).
GECCO '25 Companion, July 14–18, 2025, Malaga, Spain
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1464-1/2025/07
https://doi.org/10.1145/3712255.3726682

The BTPP takes a set of tasks and machines as input, and the goal is to assign the tasks to machines, while a machine's load must not exceed a given maximum, but should also reach a given minimum. Furthermore, there are restrictions on the compatibility between tasks and machines, i.e., not each task can be assigned to each machine. Depending on the input, not all tasks can always be planned without exceeding a machine's maximum load. Therefore, the plan can be divided into independent planning periods. There are three optimization objectives: First, the loads of machines within a planning period should be balanced. Second, each machine's load should be as close as possible to a given target value. Third, priorities should be respected, and tasks with similar priorities should be assigned to the same period. In this way, the human planner can prioritize periods containing tasks with high priority over periods with low-priority tasks.

Related problems have already been investigated in the literature. Both the Production Leveling Problem [15, 16] and the Balanced Academic Curriculum Problem [2, 3] aim to find assignments to periods while balancing the load of each period. However, relevant differences in the definition of constraints and objectives motivate the introduction of the BTPP in this paper.

The main contributions of this paper are:

- We provide an informal problem description and mathematical definition of the BTPP.
- The exact solution approaches are evaluated.
- As a metaheuristic approach, a local search method based on (Reactive) Tabu Search is proposed that can solve large-scale instances in a reasonable time. Additionally, we provide an instance generator to generate diverse instances.
- We conducted extensive experiments and compared our methods. The proposed metaheuristic methods provide high-quality solutions for the majority of large-scale instances that could not be solved by exact methods.

Full details about all proposed methods, the experimental evaluation and the NP-completeness proof can be found in [11].

2 Problem Description

The input of the BTPP consists of a set of tasks and machines. Each task has a size, a priority, and a set of eligible machines. The goal is to assign each task to a machine so that the machine loads lie within a given range. Further, a solution to the problem consists of task assignments for multiple independent planning periods. The number of periods is not predetermined, but chosen during the solution process.

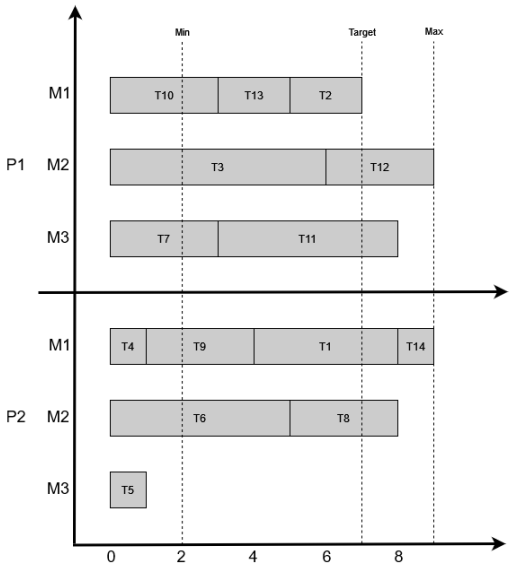

Figure 1: A candidate solution to an example instance of the BTPP

Three objective aims are calculated independently for each planning period:

- The first goal is to balance the sum of task sizes assigned to the machines within a period.
- Secondly, the load of each machine should be as close as possible to a given target load.
- Finally, the priorities of tasks assigned to the same period should be as homogeneous as possible.

As an example, consider an instance with tasks T1-T14 and machines M1-M3, where all tasks can be planned on each machine. Further, we set the minimum and maximum machine loads to 2 and 9, while the target machine load is set to 7. Figure 1 shows a candidate solution for this instance, consisting of two planning periods. In period P1, the loads of all machines lie between the minimum and maximum sizes defined by the instance. The same is not true for period P2 because task T5 only has a size of 1.

3 Complexity Analysis & Constraint Modeling Approach

We prove that the decision variant of the BTPP, which asks whether there exists a solution that satisfies all the problem's constraints (without consideration of the objective function), is NP-complete by providing a polynomial-time reduction from the Bin Packing Problem [8]. Further, we provide a constraint model that can be used with CP solvers as an exact approach to the BTPP. The full formal problem specification, the NP-completeness proof, and constraint model can be found in [11].

4 Construction Heuristic

We propose a construction heuristic that is used to generate solutions to instances of the BTPP. The main idea is to iteratively assign tasks to a period and machine. In each iteration, we select a task and then determine the machine with the least load to which the task can be assigned without violating any constraints. If no such machine exists, a new period is created to which the current task is assigned.

5 A Tabu Search Approach for the BTPP

We further propose a metaheuristic method based on Tabu search to solve the BTPP.

5.1 Solution Representation

We represent candidate solutions using a list of planning periods, where each period assigns a list of machines and each machine assigns a list of tasks. Further, we store the index of the remainder period in a separate integer variable.

During the search process, we ensure that the constraints enforcing the maximum load and the task-machine compatibilities are always satisfied. However, the constraints concerning the minimum machine load can be violated. In case of a violation, we add a penalty to the cost function $f_{\mathcal{L}}$ multiplied by a bigM value.

The constant bigM is chosen so that a single violation results in higher costs than any feasible solution. To achieve that in our implementation, we sum up worst-case values for all three objectives and increment the resulting cost by one.

5.2 Neighborhood Structure

We propose the following neighborhood move operators to explore the neighborhoods of a given solution candidate:

Shift Task Neighborhood. A *Shift Task Move* shifts a task t from its current machine and period to a given target machine m and period p. Note that a shift move may also only change the assigned machine (or period) without changing the period (or machine).

A special occurrence of a *Shift Task* move is the shift of the last remaining task in a period to another period. In this case, the period, which is empty after the move is applied, is removed from the solution. Further, if a task is shifted to a new period that is not yet part of the solution, it will be added to the list of periods.

Swap Tasks Neighborhood. A *Swap Task Move* exchanges the positions of two tasks t_1 and t_2. This type of move is only applicable if the two tasks t_1 and t_2 are not assigned to the same period and machine (i.e., $m_1 = m_2$ and $p_1 = p_2$).

5.3 Tabu Search

To avoid getting stuck in local optima, we further use a variant of Tabu Search (TS) [5]. The main idea behind TS is to avoid returning to parts of the search space that have already been seen, by keeping track of the search history using a *tabu lists*. A *tabu tenure* parameter further specifies the length of the tabu list. This parameter controls the number of iterations in which stored attributes are tabu. We propose to use two tabu list variants to solve the BTPP:

Tabu List 1: Fix Task. Here the idea is to consider the tasks affected by neighborhood moves that are used to reach a solution. Whenever a move is applied, the modified tasks are added to the tabu list. For a *SwapTasks* move, this means that both tasks are

made tabu. As long as a task is part of the tabu list, the assignments of this task to a machine and period cannot be changed.

Tabu List 2: Forbid Task Source. Here we consider the tasks affected by an applied neighborhood move as follows: Let t be a task, and m_0 and p_0 the machine and period to which t is assigned before applying the move. After applying the move, we add the tuple (t, m_0, p_0) to the tabu list, which means that a move is considered tabu if t would be moved again to machine m_0 and period p_0. Thus, we forbid the task to return to the previously assigned position.

Finally, we use an *aspiration criterion* that allows moves to be selected even if they are tabu, in case a solution is reached that has lower costs than the previously overall best cost solution.

5.4 Reactive Tabu Search

The performance of TS can vary depending on the given instance and chosen tabu tenure parameter, which defines the number of iterations a solution remains in the tabu list [13]. An extension of TS called Reactive Tabu Search (RTS) [1] aims to dynamically adapt the tabu tenure during the search process to achieve an optimized performance. This is done by keeping track of the solutions found during the search process, and whenever a solution is revisited, the tabu tenure is increased. If no solutions are visited multiple times for a number of iterations, the tabu tenure is decreased. This approach does not require setting a tabu tenure parameter manually, and further can dynamically adjust it to a value beneficial for the currently explored part of the search space. Another component of RTS is a more radical strategy to escape local optima. Although the tabu list can help to escape short cycles in the search, it often struggles with larger cycles. Therefore, a sequence of random moves is applied as a *diversification strategy* whenever the number of cycles encountered exceeds a certain threshold.

5.5 Restrict Neighborhood

In each iteration, standard TS algorithms explore the entire neighborhood to find the best neighbor. This can become computationally demanding, especially for large-scale instances. Thus, we investigate two approaches to address this issue:

Only explore a fraction of neighborhood. Every time we have to search for the best neighbor in the neighborhood, we consider each neighbor only with a probability F_{exp}, which is a parameter used to control the fraction of the neighborhood that should be explored.

Elite Candidate List. Another way to restrict the number of evaluated solutions is to use a candidate list [6]. In the first iteration, the entire neighborhood is explored and the best k neighbors are added to the candidate list. In subsequent iterations, only the moves stored in the candidate list are used to generate the neighborhood. The best move is then removed from the candidate list and applied to the current solution.

6 Experimental Evaluation

We evaluated all proposed methods using an extensive series of experiments. Using a random instance generator we developed for the BTPP we created a total of 345 problem instances, which we divided into small, medium, and large instance categories. Small instances contain between 20 and 200 tasks, medium instances between 201 and 600, and large instances between 601 and 1000 tasks.

Feasible Solutions				
	Gurobi	OR-Tools	Coin	Chuffed
Small	86%	99%	27%	62%
Medium	7%	47%	0%	4%
Large	0%	15%	0%	0%
Optimal Solutions				
	Gurobi	OR-Tools	Coin	Chuffed
Small	24%	19%	9%	3%
Medium	1%	1%	0%	0%
Large	0%	0%	0%	0%

Table 1: Percentages of instances for which a feasible/optimal solution was found using the exact/metaheuristic methods

	HC	TS	RTS	RTS-RN
Small	97%	99%	100%	100%
Medium	96%	98%	98%	98%
Large	94%	97%	97%	98%

Table 2: Percentages of instances for which a feasible solution was found using the evaluated metaheuristics.

All experiments, including parameter tuning runs, were executed on a computation cluster with 13 nodes, each with 20.48 GB RAM and an Intel(R) Xeon(R) CPU E5-2650 v4 @ 2.20GHz processor with 24 cores. Each algorithm run, both for the exact and metaheuristic methods, was executed using a wall-clock time limit of 10 minutes.

6.1 Evaluation of the Exact Methods

We implemented the constraint model using the solver-independent modeling language MiniZinc [12] and conducted experiments using the solvers Gurobi 11.0.3 [7], Google OR-Tools CP-SAT 9.10 [14], Coin-OR 2.10.11 [10] and Chuffed 0.13.2. [4]. Table 1 shows for each solver the percentages of obtained feasible/optimal instances. Feasible solutions to almost all small instances could be found using CP-SAT. Gurobi found feasible solutions to 86 small instances, followed by Chuffed with 62 and Coin-OR with 27 solved instances. For the medium-sized problem instances, only OR-Tools could find feasible solutions for nearly half of the instances, while Gurobi and Chuffed managed to find solutions only for a small portion of the instances, and Coin could not find any solution. Regarding the large-scale problems, only CP-SAT was able to obtain feasible solutions for 15 instances. Gurobi and CP-SAT obtained the largest number of optimal solutions.

6.2 Evaluation of the Metaheuristic Methods

In this section, we evaluate the performance of a basic hill-climbing algorithm (HC), standard Tabu Search (TS), Reactive Tabu Search (RTS), and Reactive Tabu Search with restricted neighborhood exploration (RTS-RN). To find high-quality parameter settings for all methods, we conducted automated parameter configuration using irace 3.5 [9].

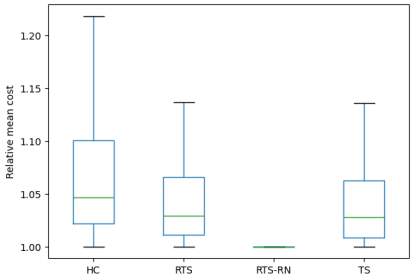

Figure 2: Boxplots showing the relative mean cost for the metaheuristic methods on the 94 large instances.

	Gurobi	CPSAT	Chuffed	Coin	HC	TS	RTS	RTS-RN
Small	44%	47%	9%	3%	4%	24%	28%	49%
Medium	2%	26%	0%	0%	1%	6%	15%	57%
Large	0%	0%	0%	0%	0%	4%	3%	94%

Table 3: Percentages of instances for which the best known solution was found by the single solution approaches

Table 2 shows the number of instances for which a feasible solution was found. The results show that most of the instances could be solved by the metaheuristics. To compare the cost results of the individual methods, we calculate the relative mean costs for each instance by dividing the mean objective value for each method by the overall best mean objective value achieved for this instance.

The results for the 94 large instances can be seen in the boxplots in Figure 2 (without outliers). As expected, the restriction of the search neighborhood has the most effect on large instances, and the results show that RTS-RN obtains significantly improved results (note that the notches, which represent the 95% confidence interval around the medians, do not overlap). Furthermore, it can be observed that the solutions found by RTS and TS are of similar quality.

6.3 Overall comparison

In this section, we compare the results of proposed methods. For the metaheuristic methods, we consider the best results found over all of the five runs. Although for most instances, a feasible solution could be found, none of the proposed methods was able to obtain solutions for instances *medium_26*, *medium_86*, and *large_10*. Further, a feasible solution to instance *large_69* could only be found using RTS-RN in one of the five algorithm runs.

Table 3 shows for each solution method the percentages of instances for which the best known solution could be found. These results show that RTS-RN provides the largest number of best upper bounds. For the small instances, CP-SAT and Gurobi showed a similar performance regarding the number of best known solutions found.

7 Conclusions

In this paper, we have introduced the Balanced Task Planning Problem We evaluated the exact approaches using state-of-the-art exact solvers. To tackle large-scale instances, we proposed a construction heuristic and Tabu Search approaches. Additionally, we investigated two strategies to reduce the size of the neighborhood explored in each iteration. Experimental results show that exact approaches could produce the best results for small instances and solve 24% of the instances to optimality. The proposed Tabu Search variants successfully obtained high-quality solutions for the large majority of medium-sized and large-scale problem instances. The experiments further show that Reactive Tabu Search that uses a restricted neighborhood can significantly improve results on realistically sized instances compared to the standard variant of Tabu Search.

Future work could examine further metaheuristic approaches for the Balanced Task Planning Problem (e.g. Genetic Algorithms or Simulated Annealing).

Acknowledgments

The financial support by the Austrian Federal Ministry for Digital and Economic Affairs, the National Foundation for Research, Technology and Development and the Christian Doppler Research Association is gratefully acknowledged.

References

[1] Roberto Battiti and Giampietro Tecchiolli. 1994. The Reactive Tabu Search. *ORSA Journal on Computing* 6, 2 (May 1994), 126–140. doi:10.1287/ijoc.6.2.126
[2] Carlos Castro and Sebastian Manzano. 2001. Variable and Value Ordering When Solving Balanced Academic Curriculum Problems. arXiv:cs/0110007
[3] Marco Chiarandini, Luca Di Gaspero, Stefano Gualandi, and Andrea Schaerf. 2012. The Balanced Academic Curriculum Problem Revisited. *Journal of Heuristics* 18, 1 (Feb. 2012), 119–148. doi:10.1007/s10732-011-9158-2
[4] G. Chu, P.J. Stuckey, A. Schutt, T. Ehlers, G. Gange, and K. Francis. [n. d.]. Chuffed, a Lazy Clause Generation Solver. https://github.com/chuffed/chuffed.
[5] Fred Glover. 1986. Future Paths for Integer Programming and Links to Artificial Intelligence. *Computers & operations research* 13, 5 (1986), 533–549.
[6] Fred Glover and Manuel Laguna. 1997. *Tabu Search.* Springer US, Boston, MA. doi:10.1007/978-1-4615-6089-0
[7] Gurobi Optimization, LLC. 2024. Gurobi Optimizer Reference Manual. https://www.gurobi.com.
[8] Juris Hartmanis. 1982. Computers and Intractability: A Guide to the Theory of NP-Completeness (Michael R. Garey and David S. Johnson). *Siam Review* 24 (1982), 90–91.
[9] Manuel López-Ibáñez, Jérémie Dubois-Lacoste, Leslie Pérez Cáceres, Mauro Birattari, and Thomas Stützle. 2016. The Irace Package: Iterated Racing for Automatic Algorithm Configuration. *Operations Research Perspectives* 3 (2016), 43–58. doi:10.1016/j.orp.2016.09.002
[10] R. Lougee-Heimer. 2003. The Common Optimization INterface for Operations Research: Promoting Open-Source Software in the Operations Research Community. *IBM Journal of Research and Development* 47, 1 (Jan. 2003), 57–66. doi:10.1147/rd.471.0057
[11] Fabian Nagler. 2025. Solution Approaches for Balanced Task Planning and Employee Task Distribution.
[12] Nicholas Nethercote, Peter J. Stuckey, Ralph Becket, Sebastian Brand, Gregory J. Duck, and Guido Tack. 2007. MiniZinc: Towards a Standard CP Modelling Language. In *Principles and Practice of Constraint Programming – CP 2007*, Christian Bessière (Ed.). Vol. 4741. Springer Berlin Heidelberg, Berlin, Heidelberg, 529–543. doi:10.1007/978-3-540-74970-7_38
[13] Paola Pellegrini, Franco Mascia, Thomas Stützle, and Mauro Birattari. 2014. On the Sensitivity of Reactive Tabu Search to Its Meta-Parameters. *Soft Computing* 18, 11 (Nov. 2014), 2177–2190. doi:10.1007/s00500-013-1192-6
[14] Laurent Perron and Frédéric Didier. 2024. CP-SAT. Google.
[15] Johannes Vass. 2019. *Exact and Metaheuristic Approaches for the Production Leveling Problem.* Thesis. doi:10.34726/hss.2019.70283
[16] Johannes Vass, Marie-Louise Lackner, Christoph Mrkvicka, Nysret Musliu, and Felix Winter. 2022. Exact and Meta-Heuristic Approaches for the Production Leveling Problem. *Journal of Scheduling* 25, 3 (June 2022), 339–370. doi:10.1007/s10951-022-00721-1